Kommandos – Sonderzeichen der Shell

BITLC BUSINESS IT LEARNING CENTER

* - steht für alle Zeichen außer für . am Namensanfang; auch kein Zeichen

? - genau 1 beliebiges Zeichen , außer . am Namensanfang,
wenn er nicht alleine auftritt

[….] - genau 1 Zeichen aus der gegeben Zeichenmenge, außer . am Namensanfang
Schreibweisen: [abcdABCD123], [a-k1-4L-Z-], []], [[….]
vordefinierte Zeichenmengen [[:alpha:]], [[blank]], [[:….:]]

[^…] - genau 1 Zeichen aus der NICHT angegebenen Zeichenmenge,
ansonsten wie […], außer . am Namensanfang

{dat..,V…xyz} - erlaubt mehrere Dat/Verz. Namen mit obigen Sonderzeichen
Achtung: keine Leerzeichen

Shell: Kommando Ende Zeichen

BITLC BUSINESS IT LEARNING CENTER

| - trennt Kommandos abhängig voneinander
stdout des Kommandos vor der Pipe wird verknüpft mit stdin des Kommandos nach der Pipe
Ausführung: quasi gleichzeitig

; - Kommandos werden nacheinander ausgeführt, sind unabhängig voneinander
Ausführung: nacheinander

& - Kommandos sind unabhängig voneinander
Ausführung: quasi gleichzeitig

Shell: Kommando Ende Zeichen

BITLC BUSINESS IT LEARNING CENTER

cmd1 && cmd2 RS: =0
verkürztes if: arbeitet cmd1 korrekt, dann wird cmd2 ausgeführt

cmd1 || cmd2 RS:>0
verkürztes if: arbeitet cmd1 fehlerhaft, dann wird cmd2 ausgeführt

cmd1 && cmd2 || cmd3
cmd 1 korrekt : cmd2 wird ausgeführt
cmd 1 fehlerhaft: cmd3 wird ausgeführt
cmd1korrekt, cmd2 fehlerhaft: cmd3 wird ausgeführt

Korrekter Befehl: Rückgabestatuswert(RS) = 0
Fehlerhafter Befehlt: RS > 0
Test: direkt nach der Kommandoausführung mit echo $?

Shell: Entwertungszeichen

Entwertungszeichen entziehen Sonderzeichen ihre Sonderbedeutung

\ - entzieht dem nachfolgenden Zeichen die Sonderbedeutung

‘ ‘ - entzieht allen Zeichen die Sonderbedeutung

“ “ - ist ähnlich wie ‘ ‘, aber die Sonderbedeutung bleibt bestehen bei:
\x, $var Variablensubstitution
$(cmd1; cmd2) Kommandosubstitution

Shell: Variablen- und Kommandosubstitution

BITLC BUSINESS IT LEARNING CENTER

Variablensubstitution

$VariableName
${VarName}xyz

dort wo das Dollarzeichen steht, wird der Inhalt der Variable ausgegeben

Kommandosubstitution

$(cmd1; cmdx …)

das Ergebnis von stdout der zwischen den Klammern stehenden Kommandos wird ausgegeben

Shell: Klammern

A. ( ) **B. [ ]** **C. { }**

A. 1) (cmd1; cmd2…) alle Kommandos in ( ) werden wie ein Kommando gesehen u. in einer Subshell ausgeführt

A. 2) $(cmd1; cmd2…) Kommandosubstitution

B. [ ] [^] Datei-Verz.-Expansion, Filterzeichen

C. 1) { cmd1;cmd2 ;} wie A. 1) allerdings Ausführung in der aktuellen Shell
Leerzeichen vor cmd1!!!

C. 2) ${VarName} Variablensubstitution

C. 3) ls /etc/{a*,d*,x*} Datei-/Verzeichnisnamen Expansion